



# Scalable GP with hyperparameters sharing based on transfer learning for solving expensive optimization problems

Caie Hu<sup>a</sup>, Sanyou Zeng<sup>b</sup>, Changhe Li<sup>c,d,e,\*</sup>

<sup>a</sup> School of Automation and Information Engineering, Xi'an University of Technology, Xi'an, 710048, China

<sup>b</sup> School of Mechanical Engineering and Electronic Information, China University of Geosciences, Wuhan, 430074, China

<sup>c</sup> School of Automation, China University of Geosciences, Wuhan, 430074, China

<sup>d</sup> Hubei Key Laboratory of Advanced Control and Intelligent Automation for Complex Systems, Wuhan, 430074, China

<sup>e</sup> Engineering Research Center of Intelligent Technology for Geo-Exploration, Ministry of Education, Wuhan, 430074, China

## HIGHLIGHTS

- The computational cost of full GPs (FGPs) is high with cubic complexity.
- This paper proposes a scalable GP to reduce the number of building FGPs.
- The knowledge from source FGPs is transferred to target GP by a linear predictor.
- The number of building FGPs and algorithmic optimization cost are markedly reduced.

## ARTICLE INFO

### Keywords:

Evolutionary computation  
Expensive optimization  
Gaussian process  
Transfer learning

## ABSTRACT

Surrogates are essential in surrogate-assisted evolutionary algorithms (SAEAs) for solving expensive optimization problems. Gaussian processes (GPs) are often used as surrogates for their accuracy in prediction and ability to quantify prediction uncertainty. However, calculating the inverse and determinant of the covariance matrix in GPs is computationally expensive because of its cubic complexity. To tackle the issue, this paper proposes a scalable GP with hyperparameters sharing based on transfer learning. A linear predictor adaptively transfers the hyperparameters knowledge from the source full GPs (FGPs) to the target GP. The transfer is performed probabilistically based on the similarity between the distributions of the training data of FGPs and target GP. In this way, the number of building FGPs is significantly reduced. As a result, the optimization cost of the algorithms is also reduced. The scalable GP can be used in SAEAs to solve expensive optimization problems. The effectiveness of the proposed method is confirmed through testing on expensive benchmark problems and a real-world antenna design problem.

## 1. Introduction

Evolutionary algorithms (EAs) are effective in solving black-box optimization problems [1]. There are several classical EAs, e.g., genetic algorithms (GA) [2], evolution strategy (ES) [3], differential evolution (DE) [4], and particle swarm optimization (PSO) [5]. DE and ES are two popular groups of methods for solving continuous optimization problems. CMA-ES is effective for solving problems that are continuous and have a few decision variables. Oppositely, DE performs well when the number of decision variables is high [6]. It is difficult to precisely differentiate between high and low numbers of decision variables.

However, if the number of variables is below 20, then it is considered being low [7].

However, EAs frequently encounter the challenge of solving some problems that require expensive evaluations, such as computationally expensive numerical simulations, costly physical experiments or unavailable analysis formulas [8]. Surrogate-assisted EAs (SAEAs) are a potential method to solve these issues, as they perform a few fitness evaluations to search for global optima [9–12]. In SAEAs, surrogate models are trained using some training data to replace the fitness function and assist in the search for the optimal solution. Candidate solutions are selected based on the aggregation function and then

\* Corresponding author at: School of Automation, China University of Geosciences, Wuhan, 430074, China.

E-mail addresses: [caie\\_hu@cug.edu.cn](mailto:caie_hu@cug.edu.cn) (C. Hu), [sanyouzeng@qq.com](mailto:sanyouzeng@qq.com) (S. Zeng), [changhe.liw@gmail.com](mailto:changhe.liw@gmail.com) (C. Li).

<https://doi.org/10.1016/j.asoc.2023.110866>

Received 24 September 2022; Received in revised form 29 July 2023; Accepted 14 September 2023

Available online 21 September 2023

1568-4946/© 2023 Elsevier B.V. All rights reserved.

reevaluated with the fitness function. The surrogate is updated based on the database after adding the candidate solutions. Typically, the aggregation function is multi-modal. The fitness and some model optimization functions (e.g., the likelihood function in the GPs) often also are multi-modal. As a result, EAs are commonly used to search for the candidate solutions in the aggregation function, the optimal parameters in the model optimization function, and the solutions in optimization problems. Such as, DE is often used as a search engine in SAEAs [13,14] and CMA-ES is also usually used as an optimizer in SAEAs [15]. Thus, by combining surrogates and EAs, SAEAs can effectively solve expensive optimization problems.

Surrogates are crucial components in SAEAs and are utilized to replace fitness functions. Untested solutions are predicted and candidate solutions are selected based on surrogates [16–19]. Several surrogates can approximate the fitness landscape, including polynomial regressions (PRs) [20], Gaussian processes (GPs) [21], radial basis function networks (RBFNs) [22], support vector machines (SVMs) [23], artificial neural networks (ANNs) [24], and ensembles of these surrogates [25].

In the abovementioned surrogates, GPs, which are probability models based on Bayesian inference, are often used in SAEAs because they can provide prediction mean and error (also known as uncertainty). In SAEAs, the uncertainty of surrogates' prediction can effectively guide the search for EAs and the update of the surrogates. Therefore, GPs are very popular in assisting EAs to solve expensive single-objective [26], multi-objective [13], many-objective [27], constrained [28], and dynamic optimization problems [29]. Unfortunately, the computational cost of building GPs is the cubic time  $\mathcal{O}(N^3)$  because of the calculation of the inversion and determinant of the covariance matrix. This challenge hinders the competitiveness of GPs when there is a large amount of training data in SAEAs, especially in solving high-dimensional problems.

Fortunately, there are numerous methods to conquer the challenge of GPs. The methods can be categorized into two categories: global approximation and local approximation [30]. Global approximation focuses on approximating the full covariance matrix. It replaces the full covariance matrix with a smaller, sparse, or low-rank covariance matrix. Local approximation focuses on dividing training data into several clusters and then building GPs in each cluster with a smaller set of training data.

The methods abovementioned primarily aim to approximate the structure of the full covariance matrix. However, GPs still need to be built once the training data is updated in these methods. Moreover, these methods present several other challenging issues: (1) The global approximation will lose some knowledge regarding the full covariance matrix; (2) The local approximation requires the construction of multiple GPs each time. It still incurs costs, unless parallel or distributed computation is used; (3) More importantly, the source knowledge of the GPs built is discarded during the optimization progress.

Like parameter transfer in transfer learning, we can transfer shared parameters or priors across tasks when the source and target tasks share some parameters or prior distribution of model hyper-parameters [31–33]. Regarding the challenge and limitations of existing methods in GPs, this paper's motivation is: When there is a high similarity between the distributions of the training data, the hyperparameters knowledge from source full GPs<sup>1</sup> (FGPs, known as source tasks) are adaptively transferred to the target GP<sup>2</sup> (known as the target task) by a linear predictor. The transfer is performed with a probability according to the similarity between the distributions of the training data. Thus, the number of building FGPs in optimization progress can be reduced, thereby the cost of algorithm optimization also can be reduced.

When SAEAs converge to a global or local optimum region, the distribution of training data tends to be similar in each iteration. During the

optimization progress of SAEAs, it is often observed that the distribution of training data after adding candidate solutions is quite similar to the distribution of the current training data, since the candidate solutions may closely resemble the samples in the current training data. As a result, the training data distributions are nearly the same, so the two GPs fitted to the training data might also be similar. Hence, our motivation is to leverage the similarity between training data distributions to allow the target GP to learn from the source FGPs, without relying solely on maximum likelihood estimation (MLE) for building the target GP. Consequently, the number of required FGPs can be reduced, and the optimization cost of the algorithms is also reduced.

The scalable GP (scalable GP: devotes to improving the scalability<sup>3</sup> of full GP while retaining the good prediction quality for big training data) with hyperparameters sharing based on transfer learning (TL-SGP) is proposed to tackle the limitations of GPs in this paper. There are two ways to learn the target GP, considering its scalability and capability. When the maximum similarity between training data distributions is less than the predefined similarity threshold, the target GP will be built using MLE. The process is time-consuming, but the target GP in this way is a FGP. The capability of the target GP can be ensured.

Otherwise, the target GP learns the knowledge of source FGPs when there is a high similarity, and the knowledge learning is based on a linear predictor. In detail, the hyperparameters of the target GP learn the ensemble of hyperparameters knowledge from the FGPs. There are two advantages to using an ensemble of hyperparameters knowledge. First, ensemble knowledge generally outperforms most individual knowledge. Second, the ensemble can generate reliable predictions across various fitness landscapes and prevent the search from getting stuck in local optima. In this way, the number of building FGPs can be reduced without sacrificing the capability of GPs.

In this paper, the TL-SGP is proposed to tackle the challenge and limitations of existing methods in GPs. The main contribution of the paper is as follows.

(1) A TL-SGP is proposed to tackle the challenge in GPs. The hyperparameters knowledge from the source FGPs is adaptively transferred to the target GP by a linear predictor. The transfer is performed with a probability according to the similarity between the distributions of the training data. In this way, the number of building FGPs is significantly reduced. As a result, the optimization cost of algorithms is also reduced.

(2) The proposed TL-SGP is used to tackle the limitations of existing methods in GPs. In this method, the source knowledge of FGPs is fully utilized in building the target GP during the optimization progress.

(3) The proposed method's effectiveness is analyzed in terms of likelihood, prediction, computational cost, and convergence. This analysis is conducted on a set of expensive benchmark problems. The experimental results show that the proposed scalable GP is promising for solving both benchmark and real-world antenna design problems.

The rest of this paper is structured as follows. Section 2 provides a review of the preliminary. Section 3 presents the proposed TL-SGP. Section 4 shows and discusses the experimental results. Finally, Section 5 concludes the paper with a summary and looks into future work.

## 2. Preliminary

The construction of GPs mainly comprises the random field assumption, hyperparameters estimation, and predictive distribution. Expected improvement (EI) is a commonly used strategy in SAEAs, which can guide the search for EAs and update of surrogates. Transfer learning is one of the prevalent methods of machine learning, which can reuse prior knowledge gained from related experiences to new tasks. This section mainly presents details of GPs, EI and transfer learning.

<sup>1</sup> Source full GPs: GPs with a full covariance matrix, which have already been built.

<sup>2</sup> Target GP: a new GP needs to be built.

<sup>3</sup> Scalability: the capability of the technique to achieve a good accuracy in prediction for the increasing number of training data..

## 2.1. Gaussian processes

Gaussian processes are widely used in regression and classification [34,35] since they can provide prediction mean and error [16,36]. Gaussian processes with the mean term of regression functions are usually used [37].

A Gaussian process as an inexpensive surrogate for a fitness function  $y = f(x)$ ,  $x = [x_1, \dots, x_D]^T \in R^D$ , is usually assumed to be a random field  $F$

$$F(x) = m(x)^T \beta + \epsilon(x), \quad (1)$$

where  $m(x) = [m_1(x), \dots, m_p(x)]$  is the basis functions at  $x$ ;  $\beta = [\beta_1, \dots, \beta_p]^T$  is the vector of coefficient,  $p$  is the number of basis functions;  $\epsilon$  with distribution  $\mathcal{N}(0, \sigma^2)$  is a random field with a known covariance structure; The correlation function  $k(x, x')$  between  $\epsilon(x)$  and  $\epsilon(x')$  is defined as (2) in this paper.

$$\hat{f}(x) \sim \mathcal{N}(K^T K^{-1} y + \gamma^T W (\sigma_\beta^2) (M^T K^{-1} y + \sigma_\beta^{-2} V^{-1} u), \hat{\sigma}^2 [1 - k^T K^{-1} k + \gamma^T W (\sigma_\beta^2) \gamma]), \quad (10)$$

$$k(x, x' | \theta) = \exp \left( - \sum_{i=1}^D \theta_i |x_i - x'_i|^2 \right), \quad (2)$$

where  $D$  is the dimension of problems;  $\theta = [\theta_1, \dots, \theta_D]^T$  measures the activity of variable  $x$ .

### 2.1.1. Hyperparameters estimation

In model (1), the hyperparameters to be estimated include  $\beta$ ,  $\theta$  and  $\sigma^2$ . The hyperparameters are usually estimated by the MLE based on observed values  $f(x_i) = y_i$  at  $x_i (i = 1, \dots, N)$ .

The log-likelihood function is

$$-\frac{1}{2} [N \log(2\pi\sigma^2) + \log(\det(K)) + (y - M\beta)^T K^{-1} (y - M\beta) / \sigma^2], \quad (3)$$

where  $K$  is a  $N \times N$  correlation matrix whose  $(i, j)$  element is  $k(x_i, x_j)$ ;  $y = [y_1, \dots, y_N]^T$  is a  $N$ -dimensional observed column vector;  $M = (m_j(x_i))$  is the known  $N \times p$  matrix of basis functions for training data.

When  $\beta$  is unknown, the estimation of  $\beta$  can be estimated by generalized least-squares

$$\hat{\beta} = (M^T K^{-1} M)^{-1} M^T K^{-1} y, \quad (4)$$

and the estimation of  $\sigma^2$  is

$$\hat{\sigma}^2 = \frac{(y - M\hat{\beta})^T K^{-1} (y - M\hat{\beta})}{N}. \quad (5)$$

Substituting (4) and (5) into (3), the simplified log-likelihood function over  $\hat{\beta}$  and  $\hat{\sigma}^2$  is

$$-N \log(2\pi\hat{\sigma}^2) - \log(\det(K)). \quad (6)$$

There is only  $\theta$  which needs to be estimated in (6). The DE (with DE/rand/1) [4] is chosen as the optimizer to estimate  $\theta$  in GP in this paper, based on the analysis in Section 4.3.

### 2.1.2. Predictive distribution

The prediction of an untested point  $x$  can be got by the conditional

distribution of  $F(x)$  given the observed values  $f(x_i) = y_i$  at  $x_i (i = 1, \dots, N)$ . There are also the following two cases.

When  $\beta$  is known, the conditional mean  $\hat{f}(x)$  is

$$m(x)^T \beta + k^T K^{-1} (y - M\beta), \quad (7)$$

where  $k$  is a known  $N \times 1$  correlation matrix between the untested point  $x$  and training data.

The conditional error  $\hat{s}^2(x)$  is

$$\hat{\sigma}^2 [1 - k^T K^{-1} k]. \quad (8)$$

When  $\beta$  is unknown, assuming the random vector  $\beta$  has prior distribution  $\mathcal{N}(\mu, \sigma_\beta^2 V)$  for some positive definite  $V$ , and  $W(\sigma_\beta^2) = (M^T K^{-1} M + \sigma_\beta^{-2} V^{-1})^{-1}$ . Then the posterior distribution of  $\beta$  given  $y$  is

$$\beta | y \sim \mathcal{N}(W(\sigma_\beta^2) (M^T K^{-1} y + \sigma_\beta^{-2} V^{-1} u), W(\sigma_\beta^2)). \quad (9)$$

The predictive distribution is

where  $\gamma = m - M^T K^{-1} k$ .

When there is a non-informative prior to  $\beta$ , it means that  $\sigma_\beta^{-2} \rightarrow \infty$ . In this case, the predictive distribution is specific. The conditional mean  $\hat{f}(x)$  is

$$m(x)^T \hat{\beta} + k^T K^{-1} (y - M\hat{\beta}), \quad (11)$$

and the conditional error  $\hat{s}^2(x)$  is

$$\hat{\sigma}^2 [1 - k^T K^{-1} k + \gamma^T (M^T K^{-1} M)^{-1} \gamma]. \quad (12)$$

In this paper, the  $\beta$  is unknown, and the basis functions  $m(x)$  consist of radial basis functions with a cubic kernel as

$$\phi(\|x - c\|) = \|x - c\|^3, \quad (13)$$

where  $c$  is a center point vector; In this paper,  $2D + 1$  basis functions are considered, and  $2D + 1$  center point vectors are got by k-means algorithm [38].

GPs can provide accurate predictions and quantify uncertainty of prediction. The two terms guide the search for algorithms to balance exploration and exploitation. However, the computational cost increases as the number of training data increases [39], where the computational complexity is  $\mathcal{O}(N^3)$ ,  $N$  refers to the number of training data. In GPs, solving the inverse and determinant of the covariance matrix is necessary for MLE. Therefore, the computational cost impedes the speed of GP-assisted EAs.

## 2.2. Expected improvement

EI is the expectation of improvement of a solution  $x$ . In minimization problems, it is monotonically decreasing with the predicted mean  $\hat{f}(x)$  and monotonically increasing with the predicted error  $\hat{s}(x)$  [16]. The improvement  $I(x)$  of a solution is defined as

$$I(x) = \max\{f_{\min} - \hat{f}(x), 0\}. \quad (14)$$

Thus, the expected improvement is

$$E[I(\mathbf{x})] = E[\max\{f_{\min} - \hat{f}(\mathbf{x}), 0\}] = (f_{\min} - \hat{f}(\mathbf{x}))\Phi\left(\frac{f_{\min} - \hat{f}(\mathbf{x})}{\hat{s}(\mathbf{x})}\right) + \hat{s}(\mathbf{x})\phi\left(\frac{f_{\min} - \hat{f}(\mathbf{x})}{\hat{s}(\mathbf{x})}\right), \quad (15)$$

where  $f_{\min}$  is the minimum value over all evaluated solutions;  $\Phi$  is the standard normal cumulative distribution function and  $\phi$  is the standard normal probability density function.

### 2.3. Transfer learning

Transfer learning is a prevalent method in machine learning, where knowledge from previous tasks (known as source tasks) can be transferred to current tasks (known as target tasks) [40–42]. Transfer learning aims to extract and apply the knowledge from source tasks to target tasks. The source and target domains usually are supposed to be related when there is an explicit or implicit relationship between their feature spaces. Transfer learning is helpful in relation-based and heterogeneous-based learning [43]. Transfer learning will be useful when knowledge exists in the form of some encoded rules, and it is easy to branch out. Moreover, when source knowledge is images collected, the target task is to classify text documents. The knowledge can still be transferred if there are relationships between the images and the text documents. In solving real-world optimization problems, if the search space has already been explored in the previous optimization, and if the previous optimization knowledge cannot be used in the following opti-

what to transfer. In this paper, hyperparameters sharing based on transfer learning is the core of the TL-SGP. In the optimization progress, the source FGPs are referred to as source tasks. Correspondingly, the target GP that needs to be built is referred to as the target task. More precisely, the hyperparameters' knowledge from source FGPs is shared with the target GP during the learning process. The ensemble hyperparameters' knowledge of source FGPs is adaptively transferred to the target GP, rather than relying on one source FGP, as shown in line 4 of Algorithm 1.

The second question needed to be answered is when to transfer. With consideration of the capability of the target GP, the learning button will be activated when there is a high similarity between training data distributions. The button will be activated when the similarity is more than or equal to the predefined similarity threshold  $ST$ , which can be set by the user preference, as shown in line 3 of Algorithm 1. Otherwise, the button will not be activated, which means the target GP will be got by MLE. In this way, the target GP got is a FGP. Notably, the accuracy of GPs is essential in optimization progress as they are used to guide the search behavior of algorithms. If GPs cannot approximate the fitness landscape of optimization problems, there will be a false optimum, which results in a false search.

---

#### Algorithm 1 Hyperparameters Sharing Based on Transfer Learning in TL-SGP

---

**Require:** Source FGPs  $S_1, \dots, S_L$ , FGPs hyperparameters  $\theta_{S_1}, \dots, \theta_{S_L}$ , current number of evaluation  $FES$ , hyperparameters space  $\Theta$ .

**Ensure:** Hyperparameters  $\theta_T$ .

- 1: Calculate similarity of two source FGPs:  $R(S_i, S_j) = \frac{1}{\sqrt{WD(S_i \| S_j)}}$ , similarity of target GP and source FGPs:  $r(T_i, S_i) = \frac{1}{\sqrt{WD(S_i \| T_i)}}$ ,  $i, j = 1, \dots, L$ ;
  - 2:  $\tau = \max(r(T_i, S_i)), i = 1, \dots, L$ ;
  - 3: if Similarity threshold  $ST \leq \tau$  then
  - 4:    $\theta_T = r^T R^{-1} \theta$ ,  $\theta = [\theta_{S_1}, \dots, \theta_{S_L}]^T$ ;
  - 5: else
  - 6:    $\theta_T$  by MLE (3),  $L = L + 1$ ;
  - 7: return Hyperparameters  $\theta_T$ .
- 

mization, it will cause the re-exploration of the similar space and increasing computational costs. The cost affects the optimization speed, especially in solving expensive optimization problems.

In transfer learning, there is source domain  $D_S$  and source task  $T_S$ , target domain  $D_T$  and target task  $T_T$ , where  $D_S \neq D_T$  or  $T_S \neq T_T$ . The purpose of transfer learning is to improve the learning of the target predictive function  $f_T$  in  $D_T$  by leveraging the knowledge from  $D_S$  and  $T_S$ . Transfer learning has been successfully used in visual categorization [44], multi-task [45], COVID-19 detection [46], and fault diagnosis [47].

### 3. Scalable GP with hyperparameters sharing based on transfer learning

This section provides details about the proposed TL-SGP. The effectiveness of the TL-SGP is investigated on the Gaussian process-assisted evolutionary algorithm framework (also known as the efficient global optimization (EGO) framework).

#### 3.1. Hyperparameters sharing based on transfer learning

The first question that needs to be answered in transfer learning is

##### 3.1.1. Similarity metric

Some common metrics can measure the distance between the two distributions. Kullback–Leibler divergence [48] is usually used to measure the distance between the two distributions. However, there is an asymmetry gap in the Kullback–Leibler divergence. The Jensen–Shannon divergence [49] was proposed to address this gap. However, the triangle inequality cannot be satisfied in the Jensen–Shannon divergence. Wasserstein distance [50] is the most effective way of measuring distance between two distributions, surpassing Kullback–Leibler and Jensen–Shannon divergence. Therefore, this paper uses the Wasserstein distance to measure the distance between the two distributions.

The Wasserstein distance, as a mathematical tool based on statistical inference, is inspired by the optimal transportation problem. In general, the second-order Wasserstein distance is commonly used to measure the distance between the two distributions. In this paper, we assume that the training data is subject to Gaussian distribution. There are two Gaussian distributions  $S_i \sim \mathcal{N}(\mathbf{u}_1, \Sigma_1)$  and  $S_j \sim \mathcal{N}(\mathbf{u}_2, \Sigma_2)$ . The mean  $\mathbf{u}$  can be estimated according to the training data ( $x_i, i = 1, \dots, N$ )

$$\mathbf{u} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i, \quad (16)$$

**Table 1**  
Test problems.

| Problem | Objective function             | $f^*$ | Property                            | Search range    |
|---------|--------------------------------|-------|-------------------------------------|-----------------|
| F1      | Shifted Sphere                 | 0     | Unimodal                            | $[-20, 20]^D$   |
| F2      | Shifted Ellipsoid              | 0     | Unimodal                            | $[-20, 20]^D$   |
| F3      | Shifted and Rotated Ellipsoid  | 0     | Unimodal                            | $[-20, 20]^D$   |
| F4      | Shifted Step                   | 0     | Unimodal, Discontinuous             | $[-20, 20]^D$   |
| F5      | Shifted Ackley                 | 0     | Multi-modal                         | $[-32, 32]^D$   |
| F6      | Shifted Griewank               | 0     | Multi-modal                         | $[-600, 600]^D$ |
| F7      | Shifted and Rotated Rosenbrock | 0     | Multi-modal with very narrow valley | $[-20, 20]^D$   |
| F8      | Shifted and Rotated Rastrigin  | 0     | Very complicated multi-modal        | $[-20, 20]^D$   |

and the empirical covariance matrix  $\Sigma$  is

$$\Sigma = \frac{\sum_{i=1}^N (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T}{N-1}, \quad (17)$$

where  $\mathbf{x}_i$  is the sample vector of training data;  $\bar{\mathbf{x}}$  is the corresponding mean vector of this training data, as shown in (16).

The second-order Wasserstein distance of two Gaussian distributions is

$$\begin{aligned} WD(S_i || S_j) &= W_2(\mathcal{N}(\mathbf{u}_1, \Sigma_1), \mathcal{N}(\mathbf{u}_2, \Sigma_2)) \\ &= \|\mathbf{u}_1 - \mathbf{u}_2\|_2^2 + Tr \left( \Sigma_1 + \Sigma_2 - 2 \left( \Sigma_1^{\frac{1}{2}} \Sigma_2 \Sigma_1^{\frac{1}{2}} \right)^{\frac{1}{2}} \right). \end{aligned} \quad (18)$$

Since  $\Sigma_1$  and  $\Sigma_2$  are symmetrical positive semi-definite matrix. It holds that  $Tr(\Sigma_1^{\frac{1}{2}} \Sigma_2 \Sigma_1^{\frac{1}{2}}) = Tr(\Sigma_1 \Sigma_2)$ , then

$$\begin{aligned} &Tr \left( \Sigma_1 + \Sigma_2 - 2 \left( \Sigma_1^{\frac{1}{2}} \Sigma_2 \Sigma_1^{\frac{1}{2}} \right)^{\frac{1}{2}} \right) \\ &= Tr(\Sigma_1) + Tr(\Sigma_2) + 2Tr \left( \left( \Sigma_1 \Sigma_2 \right)^{\frac{1}{2}} \right) \\ &= Tr \left( \left( \Sigma_1^{\frac{1}{2}} - \Sigma_2^{\frac{1}{2}} \right)^2 \right) \\ &= \left\| \Sigma_1^{\frac{1}{2}} - \Sigma_2^{\frac{1}{2}} \right\|_{Frobenius}^2. \end{aligned} \quad (19)$$

Substituting (19) into (18), the second-order Wasserstein distance is

$$WD(S_i || S_j) = \|\mathbf{u}_1 - \mathbf{u}_2\|_2^2 + \left\| \Sigma_1^{\frac{1}{2}} - \Sigma_2^{\frac{1}{2}} \right\|_{Frobenius}^2. \quad (20)$$

According to (20), the distance between two Gaussian distributions can be measured. Thus, the similarity between two Gaussian distributions can be obtained

$$r(S_i, S_j) = \frac{1}{\sqrt{WD(S_i || S_j)}}. \quad (21)$$

According to (21), the distance between two Gaussian distributions affects their similarity: a closer distance results in a higher similarity.

### 3.1.2. Linear predictor

The paper assumes that the  $\theta_T$  can be estimated by the linear regression of the  $L$  observed values of known FGPs. Assuming random vector  $\theta'_1, \theta'_2, \dots, \theta'_L, \theta'_T \sim \mathcal{N}(0, \mathbf{R})$  is Gaussian process distribution.  $\mathbf{R}$  is a known correlation matrix, and can be calculated by (21).  $\theta_1, \theta_2, \dots, \theta_L$  is realization of the random vector  $\theta'_1, \theta'_2, \dots, \theta'_L$ . The prediction of  $\theta'_T$  can be obtained by conditional distribution  $\theta'_1, \theta'_2, \dots, \theta'_L, \theta'_T$ , which is  $\theta'_T = \mathbf{r}^T \mathbf{R}^{-1} \theta'^L$ . We will prove the prediction as follows.

**Proof.** Assuming the linear predictor is

$$\theta'_T = \alpha^T \theta'^L + \varepsilon, \quad (22)$$

where  $\alpha = [\alpha_1, \alpha_2, \dots, \alpha_L]^T$ ;  $\theta'^L = [\theta'_1, \theta'_2, \dots, \theta'_L]^T$ ;  $\varepsilon$  is Gaussian noise with zero mean, and there is no correlation between  $\varepsilon$  and  $\theta'_1, \theta'_2, \dots, \theta'_L$ .

Calculate the covariance  $E[\theta'_i \theta'_T]$ , ( $i = 1, 2, \dots, L$ ) on both sides of Eq. (22), yield the following equations

$$\begin{cases} r_{1T} = E[\theta'_1 \theta'_T] = E \left[ \left( \sum_{i=1}^L \alpha_i \theta'_i + \varepsilon \right) \theta'_1 \right] \\ \quad = \alpha_1 E[\theta'^2_1] + \alpha_2 E[\theta'_1 \theta'_2] + \dots + \alpha_L E[\theta'_1 \theta'_L] + E[\varepsilon \theta'_1] \\ r_{2T} = E[\theta'_2 \theta'_T] = E \left[ \left( \sum_{i=1}^L \alpha_i \theta'_i + \varepsilon \right) \theta'_2 \right] \\ \quad = \alpha_1 E[\theta'_1 \theta'_2] + \alpha_2 E[\theta'^2_2] + \dots + \alpha_L E[\theta'_2 \theta'_L] + E[\varepsilon \theta'_2] \\ \vdots \\ r_{LT} = E[\theta'_L \theta'_T] = E \left[ \left( \sum_{i=1}^L \alpha_i \theta'_i + \varepsilon \right) \theta'_L \right] \\ \quad = \alpha_1 E[\theta'_L \theta'_1] + \alpha_2 E[\theta'_L \theta'_2] + \dots + \alpha_L E[\theta'^2_L] \\ \quad + E[\varepsilon \theta'_L]. \end{cases} \quad (23)$$

Since  $\varepsilon$  is uncorrelated to  $\theta'_1, \theta'_2, \dots, \theta'_L$ , that is,  $E[\varepsilon \theta'_i] = 0, i = 1, \dots, L$ . Eq. (23) is simplified as

$$\begin{cases} r_{1T} = \alpha_1 r_{11} + \alpha_2 r_{12} + \dots + \alpha_L r_{1L} \\ r_{2T} = \alpha_1 r_{12} + \alpha_2 r_{22} + \dots + \alpha_L r_{2L} \\ \vdots \\ r_{LT} = \alpha_1 r_{1L} + \alpha_2 r_{2L} + \dots + \alpha_L r_{LL}. \end{cases} \quad (24)$$

Eq. (24) is rewritten as a matrix form in the following

$$\mathbf{r} = \mathbf{R} \alpha. \quad (25)$$

Solve Eq. (25), yield the  $\alpha$

$$\alpha = \mathbf{R}^{-1} \mathbf{r}. \quad (26)$$

Substituting (26) into (22), yield the linear predictor is

$$\theta'_T = \mathbf{r}^T \mathbf{R}^{-1} \theta'^L. \quad (27)$$

In the TL-SGP, the hyperparameters from the source FGPs are transferred to the target GP using a linear predictor (as seen in (27)), given  $\theta_1, \theta_2, \dots, \theta_L$ . The linear predictor is

$$\theta_T = \mathbf{r}^T \mathbf{R}^{-1} \theta, \quad (28)$$

where  $\theta_T$  is the hyperparameters of the target GP;  $\mathbf{R}$  is a  $L \times L$  similarity matrix whose  $(i, j)$  element is  $r(S_i, S_j)$ ;  $\mathbf{r}$  is a known  $L \times 1$  similarity matrix of the target GP and source FGPs ( $S_1, \dots, S_L$ );  $\theta = [\theta_1, \theta_2, \dots, \theta_L]^T$ ;  $L$  is the current number of source FGPs.

When the distribution of the target GP is equal to the distribution of the source model  $S_j$ ,  $\mathbf{r}$  in (28) will be equivalent to the  $j$ th column of  $\mathbf{R}$ , which can be denoted as  $\mathbf{R}_j$ . Hence,

**Table 2**

Comparing the average fitness values (shown as Avg  $\pm$  Std) of proposed TL-SGP on DE (DE/rand/1/bin, DE/best/1/bin) and CMA-ES. Statistically significant results are evaluated using a Friedman test ( $p < 0.05$ ).

| Problem      | D  | DE/rand/1/bin                    | DE/best/1/bin                | CMA-ES                       |
|--------------|----|----------------------------------|------------------------------|------------------------------|
| F1           | 2  | <b>1.1564E-5</b> $\pm$ 1.6163E-5 | 2.7125E-3 $\pm$ 2.2916E-3    | 1.7751E-4 $\pm$ 3.3683E-4    |
|              | 5  | <b>0.0022</b> $\pm$ 0.0034       | 1.6403 $\pm$ 0.8527          | 0.0023 $\pm$ 0.0022          |
|              | 10 | 0.1560 $\pm$ 0.1184              | 35.4943 $\pm$ 12.3531        | <b>0.0248</b> $\pm$ 0.0102   |
| F2           | 2  | <b>2.3556E-5</b> $\pm$ 2.7798E-5 | 1.7710E-3 $\pm$ 2.2024E-3    | 1.4531E-4 $\pm$ 1.3520E-4    |
|              | 5  | 0.1335 $\pm$ 0.3009              | 4.5215 $\pm$ 2.5443          | <b>0.1234</b> $\pm$ 0.0959   |
|              | 10 | 14.4582 $\pm$ 8.8359             | 147.1042 $\pm$ 34.2835       | <b>5.5208</b> $\pm$ 4.0209   |
| F3           | 2  | <b>1.5327E-4</b> $\pm$ 2.0921E-4 | 1.5343E-3 $\pm$ 9.7077E-4    | 3.2895E-4 $\pm$ 3.4688E-4    |
|              | 5  | 0.4257 $\pm$ 0.4521              | 3.0487 $\pm$ 1.6956          | <b>0.3594</b> $\pm$ 0.5093   |
|              | 10 | 155.4838 $\pm$ 62.9134           | 383.5974 $\pm$ 143.8179      | <b>30.8571</b> $\pm$ 22.9017 |
| F4           | 2  | <b>0.0000</b> $\pm$ 0.0000       | <b>0.0000</b> $\pm$ 0.0000   | <b>0.0000</b> $\pm$ 0.0000   |
|              | 5  | <b>0.1200</b> $\pm$ 0.3249       | 2.0000 $\pm$ 0.7745          | 0.2000 $\pm$ 0.4000          |
|              | 10 | 2.0400 $\pm$ 1.1482              | 30.1000 $\pm$ 6.9778         | <b>0.6000</b> $\pm$ 1.1999   |
| F5           | 2  | 1.7645 $\pm$ 0.9809              | <b>1.7167</b> $\pm$ 0.5825   | 2.1789 $\pm$ 0.9570          |
|              | 5  | 2.9784 $\pm$ 0.9060              | 5.9289 $\pm$ 1.0921          | <b>2.2212</b> $\pm$ 0.5821   |
|              | 10 | 13.7498 $\pm$ 6.2305             | 12.5915 $\pm$ 1.0895         | <b>4.5811</b> $\pm$ 2.9243   |
| F6           | 2  | 0.2865 $\pm$ 0.1456              | <b>0.2425</b> $\pm$ 0.1380   | 0.2659 $\pm$ 0.2165          |
|              | 5  | 0.7174 $\pm$ 0.1454              | 0.9748 $\pm$ 0.2385          | <b>0.6752</b> $\pm$ 0.1913   |
|              | 10 | <b>0.8522</b> $\pm$ 0.1108       | 0.8779 $\pm$ 0.1881          | 1.0736 $\pm$ 0.0002          |
| F7           | 2  | 0.0111 $\pm$ 0.0123              | 0.0128 $\pm$ 0.0109          | <b>0.0051</b> $\pm$ 0.0055   |
|              | 5  | 34.4457 $\pm$ 16.7926            | <b>33.2356</b> $\pm$ 15.2143 | 36.1882 $\pm$ 14.7696        |
|              | 10 | <b>322.5220</b> $\pm$ 165.5498   | 335.7236 $\pm$ 60.3742       | 415.0965 $\pm$ 151.3114      |
| F8           | 2  | <b>0.8084</b> $\pm$ 0.7100       | 1.1104 $\pm$ 0.9550          | 1.1236 $\pm$ 0.5137          |
|              | 5  | <b>12.9111</b> $\pm$ 5.5590      | 16.8030 $\pm$ 6.7147         | 13.6635 $\pm$ 3.5842         |
|              | 10 | <b>54.7650</b> $\pm$ 7.0109      | 62.2173 $\pm$ 9.6328         | 59.4769 $\pm$ 8.9157         |
| Average rank |    | <b>1.66</b>                      | 2.54                         | 1.79                         |

$$\begin{aligned}\theta_T &= \mathbf{r}^T \mathbf{R}^{-1} \theta = (\mathbf{R}^{-1} \mathbf{r})^T \theta \\ &= (\mathbf{R}^{-1} \mathbf{R}_j)^T \theta \\ &= \mathbf{e}_j^T \theta = \theta_{S_j}.\end{aligned}\quad (29)$$

Thus, the prediction of hyperparameters  $\theta_T$  is the observed value  $\theta_{S_j}$  of source FGP  $S_j$ .

### 3.2. Gaussian process-assisted evolutionary algorithm framework

The effectiveness of the proposed TL-SGP is investigated on the Gaussian process-assisted evolutionary algorithm framework, as shown in Algorithm 2. In this framework, the initial samples are generated by Latin hypercube sampling (LHS) [51]. These samples are evaluated by the fitness function and archived into an initial database. The samples in the database are used to train a GP (e.g., FGP or TL-SGP). Then, the GP replaces the fitness function and strategy (e.g., EI) as an aggregate function to evolve a population of  $NP$  individuals for  $T$  generations with an optimizer. The candidate solution is selected and reevaluated using the fitness function. Then the database is updated. The algorithm will be performed until the computational budget is exhausted. The best solutions in the database will be outputted.

## 4. Results and discussion

The effectiveness of the proposed TL-SGP is investigated according to comparison with three algorithms: FGP obtained from MLE, typical global approximation scalable GP (GA-SGP) with a smaller covariance matrix, and local approximation scalable GP (LA-SGP) with naive local experts [30]. The experiments compare the four algorithms in terms of likelihood, prediction accuracy, convergence, and running efficiency on the CEC'2014 expensive benchmark problems [52] with 2, 5 and 10 dimensions (Notably, the comparison is performed in 2, 5 and 10 dimensions, given that the compared models (FGP and LA-SGP) involve much computational cost in high-dimensional problems). Moreover, the proposed method's effectiveness also is investigated in solving a real-world waveguide antenna design problem.

For FGP, all training data in the database is used to build the GP using MLE. For GA-SGP,  $m$  latest samples in the database are used to build the GP. For LA-SGP, several clusters are clustered by fuzzy clustering [13], and the GP is built within each cluster.

### 4.1. Test benchmark problems

The proposed TL-SGP was tested using expensive benchmark problems from CEC'2014 with varying properties. The properties of these

---

#### Algorithm 2 The pseudo-code of the Gaussian process-assisted evolutionary algorithm framework

---

**Require:** Generate initial samples by LHS and evaluate using fitness function, archive into an initial database.

**Ensure:** Best solution in the database.

**while** The computational budget  $FES$  is not exhausted **do**

    Train a GP model, e.g.,  $\triangleright$  call Algorithm , replace fitness function;

    Evolve a population of  $NP$  individuals for  $T$  generations with an optimizer, evaluate by a criterion (e.g., EI);

    Select a candidate solution to reevaluate using the fitness function;

    Add the solution to the database, and update the database,  $FES = FES + 1$ .

**return** Best solution in the database.

---

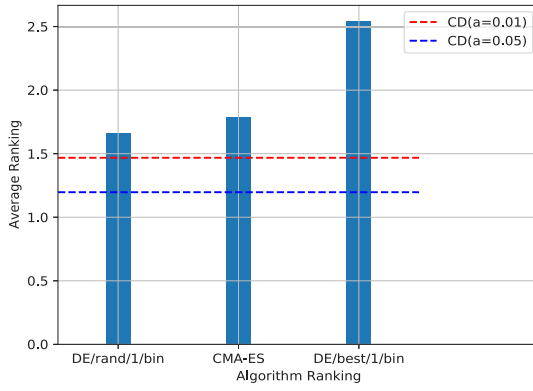


Fig. 1. Results of Friedman test and Bonferroni–Dunn with taking TL-SGP with DE/rand/1/bin as a control algorithm.

problems are described in Table 1.

#### 4.2. Parameter settings

There are several parameters in the algorithms. The setting of these parameters is given below.

- (1) The computational budget  $FEs = 100$  was performed in this paper. We assumed that the optimization algorithm is only allowed to evaluate a few candidate solutions by the fitness function during the optimization process.
- (2) LHS randomly generated initial  $11D - 1$  samples.
- (3) The range of values for parameters  $\theta$  was  $[1.0e - 6, 100]$ .

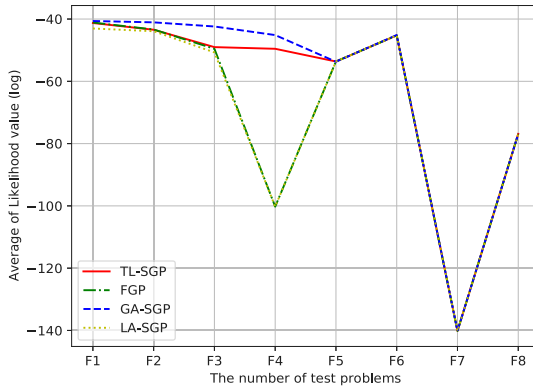


Fig. 2. Likelihood function results of FGP, TL-SGP, GA-SGP and LA-SGP on test problems.

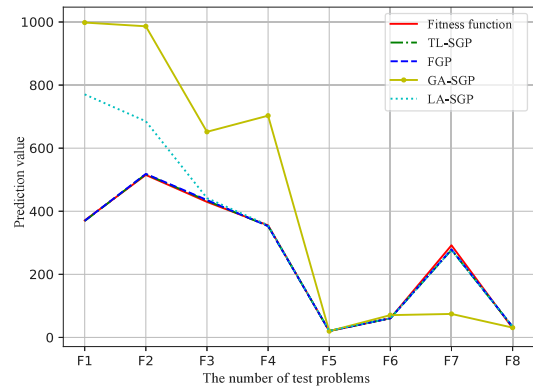


Fig. 3. Prediction function results of FGP, TL-SGP, GA-SGP and LA-SGP on test problems.

(4) The number of runs was 25.

(5) The similarity threshold (ST) was 0.9 in this paper, which could be set by the user (see literature [53]).

(6) Latest samples  $m = 11D - 1$  in the database were used in GA-SGP based on the capability and scalability of GP.

(7)  $L1 = 80$  and  $L2 = 20$  in fuzzy clustering were based on setting in the literature [13].

#### 4.3. Sensitivity analysis of optimizer

To test the effect of the proposed TL-SGP on different EA-based optimizers, two significantly different EA-based optimizers are compared on the proposed TL-SGP. One is DE with operators DE/rand/1/bin and DE/best/1/bin, and another is CMA-ES. Both optimizers have the same parameter settings: evolution generations  $T = 100$  and population size  $NP = 20$ ; DE/rand/1/bin and DE/best/1/bin: scaling factor  $F = 0.5$  and crossover rate  $CR = 0.9$  based on the research [54]. The results of the proposed TL-SGP on DE (DE/rand/1/bin, DE/best/1/bin) and CMA-ES in solving the CEC'2014 expensive benchmark problems with  $D = 2, 5, 10$  are shown in Table 2.

From Table 2, the proposed TL-SGP with DE/rand/1 outperforms other optimizers on most benchmark problems, especially in multi-modal benchmark problems with  $D = 10$ . TL-SGP algorithm with DE/rand/1/bin is ranked first and there are significant differences among these algorithms ( $p$ -value  $< 0.05$ ). This paper uses the Bonferroni–Dunn test to compare the significant level of all algorithms at  $\alpha < 0.05$ . The critical difference (CD) is,

$$CD = q \sqrt{\frac{k(k+1)}{6N}}, \quad (30)$$

where  $k$  and  $N$  are the number of algorithms and benchmark tasks, respectively. In this experimental comparison,  $k = 3$  and  $N = 8$ .

When  $\alpha < 0.01$  and  $\alpha < 0.05$ ,  $q$  is 2.936 and 2.394, respectively, referring Table B15 in [55]. There is a significant difference between these algorithms at the significant levels of  $\alpha < 0.01$  through observing the results of the Bonferroni–Dunn test in Fig. 1. The results show that the algorithm TL-SGP with DE/rand/1/bin has a better optimization performance.

For CMA-ES, it is easy to track into local optima, while the aggregation and model optimization functions are both multi-modal. The EI (see (15)) is highly multi-modal, and there are large areas where the expected improvement is essentially zero and even appears quite flat areas. These limitations make it more challenging to search for the optimum EI. Thus, there are numerous challenges for CMA-ES and DE/best/1. We will choose DE/rand/1 as the optimizer based on the issue in following comparison.

#### 4.4. Comparison on likelihood

The likelihood function can measure how well the hyperparameters fit the assumed probability distribution based on the observed training data. Therefore, the likelihood value of optimum hyperparameters can measure the probability model's effectiveness in the observed training data. If the likelihood value is high, the hyperparameters are better and the training data is more likely to appear.

In the experimental comparison on the  $D = 2$ , the average likelihood values on 25 runs are got by FGP, TL-SGP, GA-SGP and LA-SGP, respectively, as shown in Fig. 2. From this figure, the likelihood values of the four models are similar in most test problems. This case means that the optimum hyperparameters from the four models can fit the assumed probability distribution based on the observed training data. The likelihood values got from GA-SGP and the proposed TL-SGP are slightly higher than others, especially in the case of F2, F3, and F4 problems.

For GA-SGP, the latest training data is used to train the GP model

**Table 3**

Comparing the average fitness values (shown as Avg  $\pm$  Std) of FGP, GA-SGP, LA-SGP, and TL-SGP. Statistically significant results are evaluated using a Friedman test ( $p < 0.1$ ).

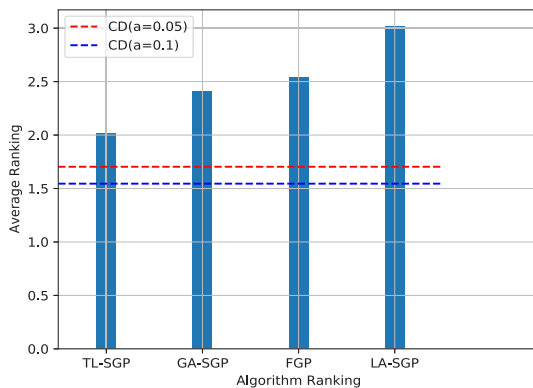
| Problem      | D  | FGP                          | GA-SGP                           | LA-SGP                     | TL-SGP                           |
|--------------|----|------------------------------|----------------------------------|----------------------------|----------------------------------|
| F1           | 2  | 0.0017 $\pm$ 0.0025          | 5.6419E-5 $\pm$ 4.5921E-5        | 3.1074E-5 $\pm$ 3.9101E-5  | <b>1.1564E-5</b> $\pm$ 1.6163E-5 |
|              | 5  | 0.0056 $\pm$ 0.0065          | <b>4.3078E-4</b> $\pm$ 3.1163E-4 | 0.0024 $\pm$ 0.0022        | 0.0022 $\pm$ 0.0034              |
|              | 10 | <b>0.0740</b> $\pm$ 0.0571   | 0.1203 $\pm$ 0.0942              | 0.3398 $\pm$ 0.3097        | 0.1560 $\pm$ 0.1184              |
| F2           | 2  | 0.0035 $\pm$ 0.0072          | 8.6589E-5 $\pm$ 8.4272E-5        | 8.6459E-5 $\pm$ 9.4072E-5  | <b>2.3556E-5</b> $\pm$ 2.7798E-5 |
|              | 5  | 0.0331 $\pm$ 0.0176          | <b>0.0315</b> $\pm$ 0.0372       | 0.1602 $\pm$ 0.1982        | 0.1335 $\pm$ 0.3009              |
|              | 10 | <b>8.1824</b> $\pm$ 4.5741   | 11.9587 $\pm$ 7.7897             | 15.4139 $\pm$ 5.1924       | 14.4582 $\pm$ 8.8359             |
| F3           | 2  | 0.0037 $\pm$ 0.0048          | 8.0356E-4 $\pm$ 8.9242E-4        | 3.2863E-4 $\pm$ 3.7119E-4  | <b>1.5327E-4</b> $\pm$ 2.0921E-4 |
|              | 5  | <b>0.1524</b> $\pm$ 0.2473   | 0.1827 $\pm$ 0.1810              | 0.6163 $\pm$ 0.2892        | 0.4257 $\pm$ 0.4521              |
|              | 10 | <b>97.9649</b> $\pm$ 54.4764 | 177.7319 $\pm$ 100.9255          | 391.0655 $\pm$ 107.7111    | 155.4838 $\pm$ 62.9134           |
| F4           | 2  | 0.0400 $\pm$ 0.1959          | <b>0.0000</b> $\pm$ 0.0000       | <b>0.0000</b> $\pm$ 0.0000 | <b>0.0000</b> $\pm$ 0.0000       |
|              | 5  | 0.0800 $\pm$ 0.2712          | <b>0.0000</b> $\pm$ 0.0000       | 0.1600 $\pm$ 0.3666        | 0.1200 $\pm$ 0.3249              |
|              | 10 | <b>1.2400</b> $\pm$ 1.0307   | 1.2800 $\pm$ 1.2496              | 2.0000 $\pm$ 1.0954        | 2.0400 $\pm$ 1.1482              |
| F5           | 2  | 2.0199 $\pm$ 1.0530          | 2.1751 $\pm$ 1.1817              | 2.1089 $\pm$ 0.9216        | <b>1.7645</b> $\pm$ 0.9809       |
|              | 5  | 3.2036 $\pm$ 0.8389          | 6.6501 $\pm$ 5.7352              | 4.1851 $\pm$ 0.9159        | <b>2.9784</b> $\pm$ 0.9060       |
|              | 10 | 13.7021 $\pm$ 6.4746         | <b>7.6967</b> $\pm$ 3.1714       | 11.7713 $\pm$ 2.6594       | 13.7498 $\pm$ 6.2305             |
| F6           | 2  | 0.3573 $\pm$ 0.2128          | 0.3294 $\pm$ 0.1862              | <b>0.2757</b> $\pm$ 0.1454 | 0.2865 $\pm$ 0.1456              |
|              | 5  | 0.7074 $\pm$ 0.1635          | 0.7198 $\pm$ 0.2230              | <b>0.6958</b> $\pm$ 0.0968 | 0.7174 $\pm$ 0.1454              |
|              | 10 | 0.8783 $\pm$ 0.1043          | <b>0.8479</b> $\pm$ 0.1881       | 1.0739 $\pm$ 0.0167        | 0.8522 $\pm$ 0.1108              |
| F7           | 2  | 0.0269 $\pm$ 0.0198          | 0.0424 $\pm$ 0.0453              | 0.0238 $\pm$ 0.0225        | <b>0.0111</b> $\pm$ 0.0123       |
|              | 5  | 40.1885 $\pm$ 17.2449        | 37.9399 $\pm$ 25.8264            | 50.8934 $\pm$ 13.5115      | <b>34.4457</b> $\pm$ 16.7926     |
|              | 10 | 350.9232 $\pm$ 87.7120       | 335.7236 $\pm$ 60.3742           | 459.1892 $\pm$ 120.1929    | <b>322.5220</b> $\pm$ 165.5498   |
| F8           | 2  | 1.1892 $\pm$ 0.8007          | 1.4791 $\pm$ 1.2482              | 1.2723 $\pm$ 1.0582        | <b>0.8084</b> $\pm$ 0.7100       |
|              | 5  | 12.7410 $\pm$ 5.4972         | <b>11.8876</b> $\pm$ 4.7678      | 18.6188 $\pm$ 4.2618       | 12.9111 $\pm$ 5.5590             |
|              | 10 | 56.3820 $\pm$ 12.0586        | 57.1793 $\pm$ 12.5400            | 58.4504 $\pm$ 10.6282      | <b>54.7650</b> $\pm$ 7.0109      |
| Average rank |    | 2.54                         | 2.41                             | 3.02                       | <b>2.02</b>                      |

based on the scalability and capability of GP. It means that GA-SGP is an approximate model with the latest training data, unlike FGP. Fig. 2 shows that GA-SGP has a higher likelihood function, especially in uni-modal problems. For uni-modal problems  $F1 - F4$ , the algorithm converges to a global optimum region easily. Therefore, it is easy for GA-SGP to fit the region's landscape.

For the proposed TL-SGP, the target GP is from FGP and SGP with hyperparameters sharing, which are adaptively switched according to the similarity between the distributions of the training data. When there is a high similarity, the SGP will be performed instead of the FGP. The switch ensures the reliability of TL-SGP. In addition, the ensemble of hyperparameters based on a linear predictor is used in this method. The ensemble of hyperparameters generally outperforms most individual hyperparameters and improves the robustness of the model.

#### 4.5. Comparison on prediction

The prediction accuracy usually is used to measure the quality of



**Fig. 4.** Results of the Friedman test and Bonferroni–Dunn with taking TL-SGP as the control algorithm.

models. The smaller the fitting error is, the better the model is.

In the experimental comparison on the  $D = 2$ , the average prediction values from 25 runs are got by FGP, TL-SGP, GA-SGP and LA-SGP, respectively, as shown in Fig. 3. This figure shows that FGP and TL-SGP have similar prediction values and can fit the fitness function in all test problems. However, LA-SGP and GA-SGP are relatively bad in prediction, especially in  $F1 - F4$  and  $F7$ .

The latest training data is only used to train GA-SGP. The GA-SGP will be a local model if the algorithm converges to a global or local optimum region. Thus, GA-SGP learns a local fitness landscape, but still needs to improve its prediction ability for the global landscape.

For LA-SGP, there are several local GPs, and the prediction of any untested point will be obtained according to the nearest local model. However, the LA-SGP are poor at fitting the global fitness landscape, which results in poor prediction.

Notably, the prediction accuracy of TL-SGP is slightly higher than FGP. The likelihood function in GP is multi-modal. Thus, the obtained hyperparameters by MLE may be the local optimum, and it is not easy to find the global optimum since there are many flat areas. The hyperparameters' ensemble knowledge is used in TL-SGP. It can help prevent trapping local optima. Therefore, the TL-SGP can be more reliable than FGP relatively.

#### 4.6. Comparison on convergence

The convergence is often used to measure the optimization ability of the algorithm. The experiment is performed on 2, 5 and 10 dimensions to verify the effectiveness of the proposed TL-SGP in solving different

**Table 4**

Homogeneity of Variance Test.

|     | Levene Statistic | df1 | df2 | Sig.  |
|-----|------------------|-----|-----|-------|
| 2D  | 0.300            | 3   | 28  | 0.825 |
| 5D  | 0.307            | 3   | 28  | 0.820 |
| 10D | 1.264            | 3   | 28  | 0.306 |

**Table 5**  
ANOVA.

|     |                | Sum of Squares | df | Mean Square | F     | Sig.  |
|-----|----------------|----------------|----|-------------|-------|-------|
| 2D  | Between Groups | 0.089          | 3  | 0.030       | 0.051 | 0.984 |
|     | Within Groups  | 16.236         | 28 | 0.580       |       |       |
|     | Total          | 16.325         | 31 |             |       |       |
| 5D  | Between Groups | 36.416         | 3  | 12.139      | 0.057 | 0.982 |
|     | Within Groups  | 5917.994       | 28 | 211.357     |       |       |
|     | Total          | 5954.409       | 31 |             |       |       |
| 10D | Between Groups | 11 634.360     | 3  | 3878.120    | 0.211 | 0.888 |
|     | Within Groups  | 514 531.686    | 28 | 18 376.132  |       |       |
|     | Total          | 526 166.045    | 31 |             |       |       |

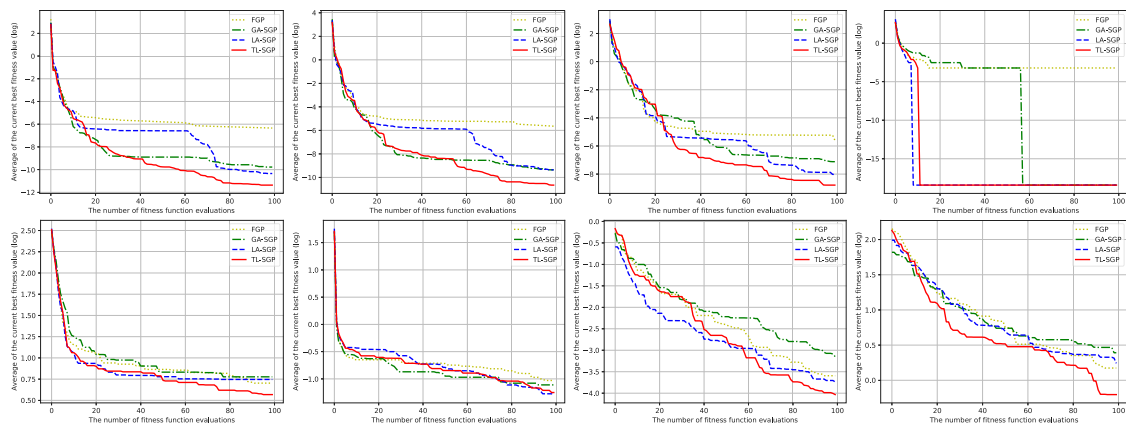
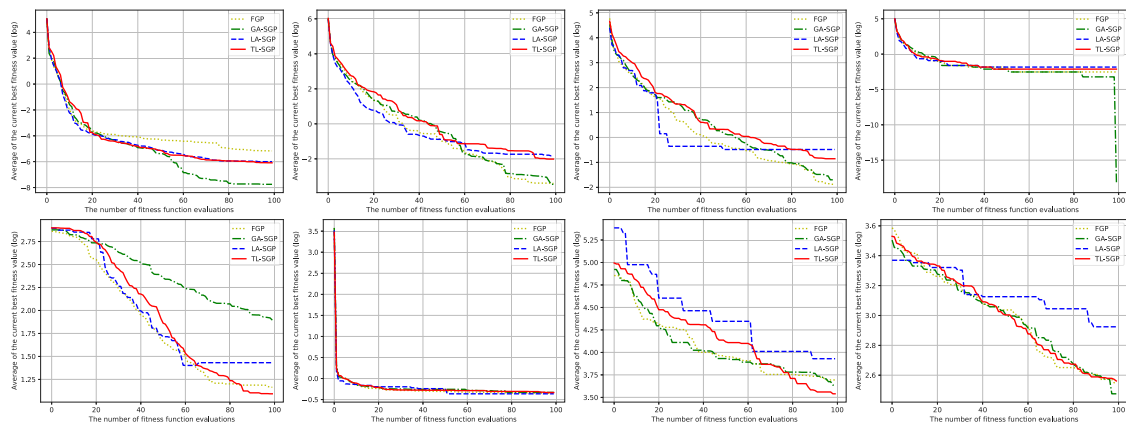
**Table 6**  
Normality Tests.

| D   | Alg.   | Kolmogorov–Smirnov |    |       | Shapiro–Wilk |    |       |
|-----|--------|--------------------|----|-------|--------------|----|-------|
|     |        | Statistic          | df | Sig.  | Statistic    | df | Sig.  |
| 2D  | FGP    | 0.334              | 8  | 0.009 | 0.694        | 8  | 0.002 |
|     | GA-SGP | 0.332              | 8  | 0.010 | 0.680        | 8  | 0.001 |
|     | LA-SGP | 0.341              | 8  | 0.007 | 0.671        | 8  | 0.001 |
|     | TL-SGP | 0.333              | 8  | 0.009 | 0.670        | 8  | 0.001 |
| 5D  | FGP    | 0.369              | 8  | 0.002 | 0.628        | 8  | 0.000 |
|     | GA-SGP | 0.312              | 8  | 0.021 | 0.639        | 8  | 0.000 |
|     | LA-SGP | 0.375              | 8  | 0.002 | 0.616        | 8  | 0.000 |
|     | TL-SGP | 0.363              | 8  | 0.003 | 0.627        | 8  | 0.000 |
| 10D | FGP    | 0.293              | 8  | 0.041 | 0.633        | 8  | 0.000 |
|     | GA-SGP | 0.318              | 8  | 0.017 | 0.693        | 8  | 0.002 |
|     | LA-SGP | 0.365              | 8  | 0.002 | 0.645        | 8  | 0.001 |
|     | TL-SGP | 0.312              | 8  | 0.021 | 0.700        | 8  | 0.002 |

dimensional problems.

Table 3 presents all algorithms' average best fitness values and statistically significant results using a Friedman test on benchmark problems with 2, 5 and 10 dimensions. The results also are statistically analyzed using a one-way analysis of variance (ANOVA) as shown in Tables 4 and 5, and using Kruskal–Wallis test as shown in Table 6.

TL-SGP algorithm outperforms FGP, GA-SGP, and LA-SGP algorithms in most test problems, particularly in multi-modal problems (see Table 3). The mean ranking from the Friedman test shows that the algorithm TL-SGP is first, and all  $p$  values are less than 0.1. Moreover, the Bonferroni–Dunn test also is used to compare the significant level of all the algorithms in  $\alpha < 0.1$ . In the comparison,  $k = 4$  and  $N = 8$ . When  $\alpha < 0.05$  and  $\alpha < 0.1$ ,  $q$  is 2.639 and 2.394, respectively (also see [55]). The Bonferroni–Dunn test results in Fig. 4 show a significant difference between these algorithms at a significance level of  $\alpha < 0.05$ . The results show that the algorithm TL-SGP has a better optimization performance.

**Fig. 5.** Convergence curves of fitness functions for F1 – F8 on problem with 2D.**Fig. 6.** Convergence curves of fitness functions for F1 – F8 on problem with 5D.

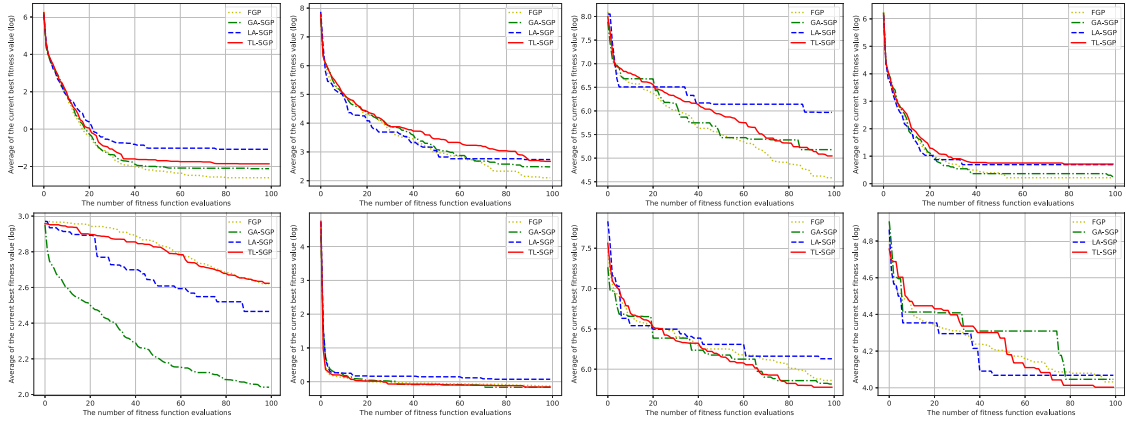


Fig. 7. Convergence curves of fitness functions for  $F1 - F8$  on problem with 10D.

According to the results in Table 4,  $p$  values are more than 0.05, indicating that the variance is homogeneous in  $D = 2, 5, 10$  results. From Table 5,  $p$  values are more than 0.05, which means that the mean is consistent in  $D = 2, 5, 10$  results. From Table 6, it can be observed that  $p$ -values are less than 0.05 in  $D = 2, 5, 10$  results, showing significant differences among the distribution of the results.

Figs. 5–7 present the comparative behaviors of four algorithms in optimization progress. These figures show that TL-SGP performs better and converges faster than FGP, GA-SGP, and LA-SGP in most benchmark problems with varying dimensions.

The GA-SGP will be a local model if the algorithm convergence to a global or local optimum region. Thus, the GA-SGP only learns approximately local fitness landscapes. The models can assist in the search for the EA, especially in uni-modal problems. However, it is not effective in

most multi-modal problems because there are many local optima.

For LA-SGP, several clusters can fit the local characters of fitness landscapes. However, the LA-SGP is poor at fitting global fitness landscapes. Thus, the LA-SGP cannot guide the global search of the algorithm well.

The proposed TL-SGP comprises FGP and SGP, which are adaptively switched according to the similarity between training data distributions. This way ensures the reliability of TL-SGP. Moreover, the proposed TL-SGP applies the ensemble of hyperparameters sharing based on a linear predictor in this paper. The ensemble knowledge can generate more reliable predictions than individual hyperparameters on various fitness landscapes (see Fig. 3). Compared to the algorithm with FGP, the algorithm with TL-SGP can prevent the search from getting stuck in local optima in multi-modal problems.

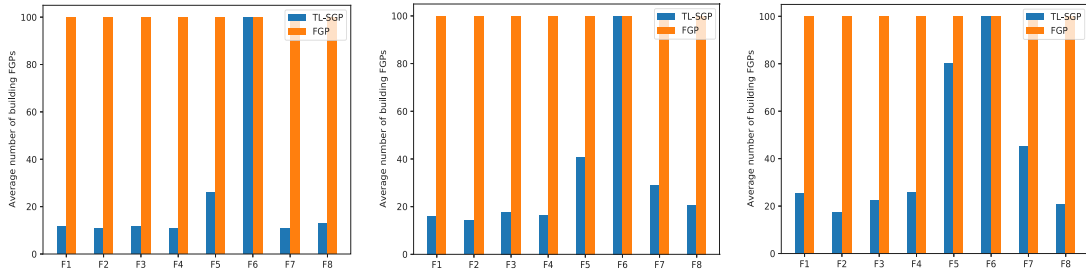


Fig. 8. The average number of building FGPs in algorithm FGP and TL-SGP on test problems of 2,5 and 10D.

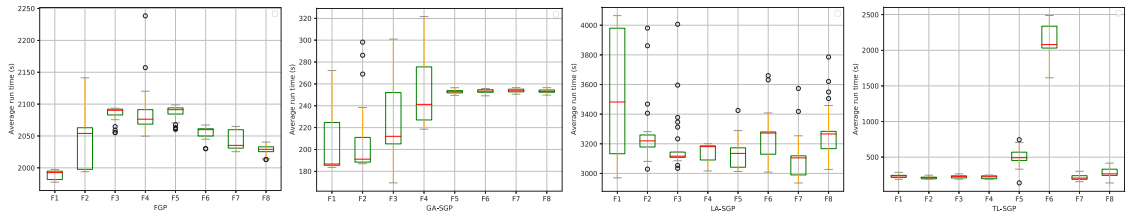


Fig. 9. Average run time of FGP, GA-SGP, LA-SGP and TL-SGP on test problems with 2D.

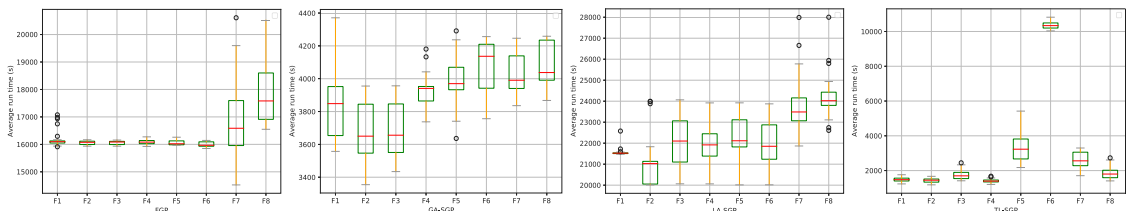


Fig. 10. Average run time of FGP, GA-SGP, LA-SGP and TL-SGP on test problems with 5D.

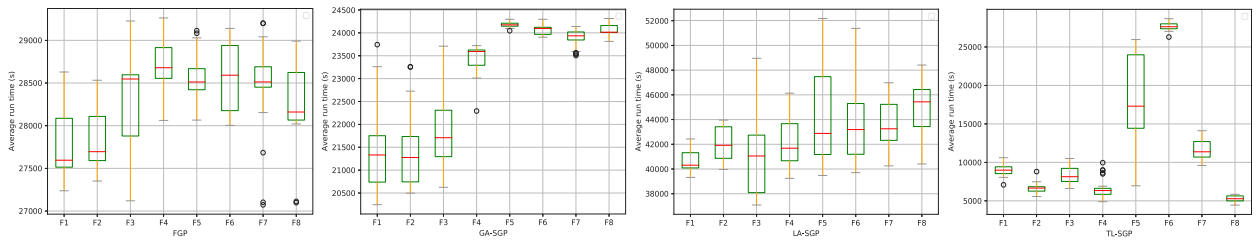


Fig. 11. Average run time of FGP, GA-SGP, LA-SGP and TL-SGP on test problems with 10D.

#### 4.7. Comparison on running efficiency

There are only different GP models in each algorithm, yet the rests are the same to ensure fairness of algorithms. The number of FGPs needed to be built in the algorithm FGP and TL-SGP is counted as shown in Fig. 8. It is clear that the number of building FGPs in TL-SGP is significantly less than in FGP in most benchmark problems. Thus, the proposed TL-SGP can reduce the number of building FGPs. However, for  $F_6$ , there is no significant difference. It can be attributed to the large search range for the  $F_6$  function (see Table 1). Therefore, there are a few highly similar (with more than 0.9) training data distributions. In this case, some FGPs will need to be built.

The run time reflects the computational cost of optimization. The less

run time is, the better the algorithm has. In the experimental comparison on 2, 5 and 10 dimensions, the experiment was run on a PC with 64-bit Inter(R) Core(TM) i5-8500 CPU of 3.00 GHZ, RAM 8GB, and Python programming language. The average run time on 25 runs is got by FGP, GA-SGP, LA-SGP and TL-SGP, as shown in Fig. 9, Fig. 10 and Fig. 11, respectively. These figures show that the runtime of the proposed TL-SGP is significantly less than the others on 2, 5 and 10 dimensions.

For FGP, calculating the covariance matrix's inversion and determinant must be performed in each solving MLE. Thus, the computation cost is unavoidably high.

For GA-SGP, the computation cost depends on the number of the training data. However, the capability of GA-SGP will be low if the model is trained with a small amount of training data.

LA-SGP incurs costs unless parallel or distributed computation is used. Parallel computation is not used in this experiment. Thus, some computational cost will still be incurred.

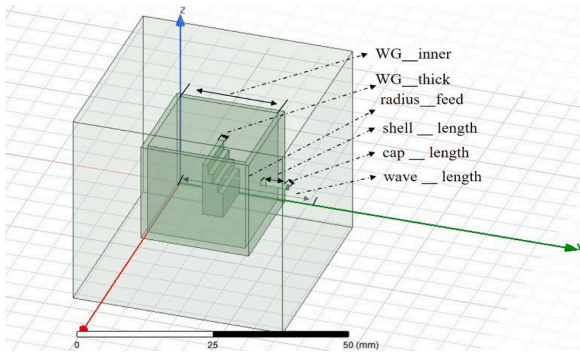


Fig. 12. Structure of waveguide antenna.

Table 7

Performance requirement of waveguide antenna design.

| Parameter                          | Specification                                                             |
|------------------------------------|---------------------------------------------------------------------------|
| Working frequency                  | 8 GHz–12 GHz                                                              |
| Voltage standing wave ratio (VSWR) | $\leq 2$                                                                  |
| Gain                               | $\geq 0$ dB, $-50^\circ \leq \theta \leq 50^\circ$ , $\varphi = 90^\circ$ |
| Polarization mode                  | Linear polarization                                                       |
| Input impedance                    | 50 $\Omega$                                                               |

#### 4.8. Real-world case study in antenna structure design

Antenna structure design involves computationally expensive simulations since Maxwell's equations must be solved. Solving the Maxwell equation involves time-consuming finite-element analysis for each frequency within the band of interest in electromagnetic simulation software (e.g., HFSS). Therefore, antenna structure design is typically an expensive optimization problem. The goal of antenna structure design is to achieve an optimal structure that meets the performance requirements of the antenna design. A waveguide antenna is designed to verify the effectiveness of the proposed method in this paper.

##### 4.8.1. Waveguide antenna structure

The waveguide antenna structure is shown in Fig. 12. The antenna consists of a polarizer, waveguide inner and outer wall, cap, feed and feed shell. There are  $D = 6$  decision variables that need to be optimized.

In Fig. 12,  $WG\_inner$  is the size of the waveguide inner wall,  $WG\_thick$  is the thickness of the polarizer,  $radius\_feed$  is the radius of feedline;  $shell\_length$ ,  $cap\_length$ ,  $wave\_length$  is the lengths of the feedline sell, cap and waveguide, respectively.

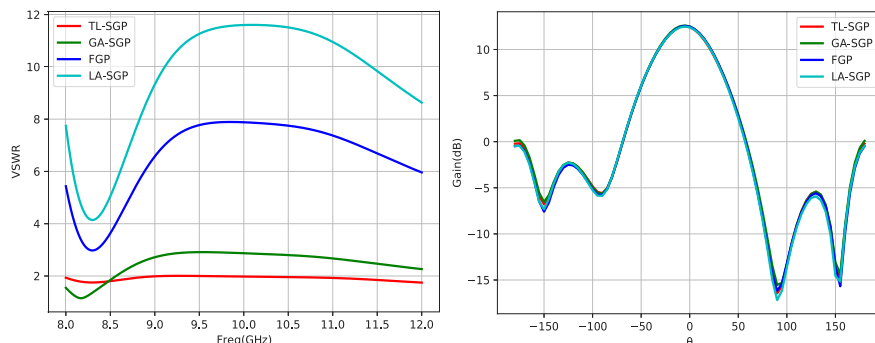


Fig. 13. The VSWR and Gain (at freq=10 GHz,  $-50^\circ \leq \theta \leq 50^\circ$ ,  $\varphi = 90^\circ$ ) results from GA-SGP, LA-SGP, FGP, and TL-SGP.

#### 4.8.2. Requirement of waveguide antenna design

Table 7 shows the waveguide antenna design performance requirements based on practical engineering needs.

#### 4.8.3. Waveguide antenna optimization problem

According to the requirement of waveguide antenna design, as shown in Table 7, the optimization problem is defined as

$$\begin{aligned} f(\mathbf{x}) &= \minmax(VSWR_{freq}), freq = 8GHz - 12GHz \\ \mathbb{X} &= \{\mathbf{x} | \mathbf{l} \leq \mathbf{x} \leq \mathbf{u}\} \\ \mathbf{x} &= (WG\_inner, WG\_thick, radius\_feed, shell\_length, cap\_length, \\ &\quad wave\_length) \\ \mathbf{l} &= (10, 0.5, 0.1, 3, 0.25, 16), unit : mm \\ \mathbf{u} &= (30, 1.5, 0.3, 8, 0.75, 45), \end{aligned} \quad (31)$$

where VSWR measures the power reflected from the antenna, which is the function of reflection coefficient;  $freq$  is the working frequency; elevation angle is  $-180^\circ \leq \theta \leq 180^\circ$ , azimuth is  $0^\circ \leq \varphi \leq 180^\circ$ , and step size is  $5^\circ$ .

#### 4.8.4. Optimization results

The experiment is run on a PC with 64-bit Inter(R) Core(TM) i5-8500 CPU of 3.00 GHz, RAM 8 GB, and the antenna is evaluated by HFSS 18.0 simulation software. The waveguide antenna is designed using four different algorithms (FGP, GA-SGP, LA-SGP and TL-SGP). All parameter settings are the same as in Section 4.2. The VSWR and Gain results from the four algorithms are shown in Fig. 13.

In engineering,  $VSWR \leq 2.5$  is usually available. From Fig. 13, we can observe that the antenna performance from the proposed TL-SGP outperforms others. The VSWR and Gain results from the proposed TL-SGP approximately satisfy the design requirement in Table 7 across the entire frequency range, which approximate  $VSWR \leq 2$  and  $Gain \geq 0$  at  $-50^\circ \leq \theta \leq 50^\circ, \varphi = 90^\circ$ . The results also indicate that the antenna optimized by the proposed TL-SGP is broadband. Although the Gain results from the other three compared algorithms satisfy the Gain requirement in Table 7, VSWR results do not satisfy the design requirement, especially for the results from LA-SGP. Therefore, the proposed TL-SGP can solve the antenna design problem. It is more competitive than the other three algorithms in solving the waveguide antenna design problem.

## 5. Conclusion

This paper focuses on tackling the challenge of building GPs and the limitations of existing scalable GPs. Hence, the TL-SGP is proposed to tackle the issues. In this method, the hyperparameters knowledge from source FGPs is adaptively transferred to the target GP by a linear predictor. In this way, the number of building FGPs can be reduced significantly, and the optimization cost of the algorithms can be reduced.

The proposed method's effectiveness has been analyzed and investigated based on expensive benchmark problems and an antenna design problem. The experimental results show the proposed method is more effective than FGP, GA-SGP, and LA-SGP on most test problems and waveguide antenna design problem.

The effectiveness of the proposed method is only investigated on problems with low dimensions. However, there are many high-dimensional problems in practice. The proposed method is verified to be effective on problems with a small search range (or small solution space), but there is a limitation on problems with a large search range. Therefore, future work will apply the proposed method to solve high-dimensional problems. More importantly, we will focus on addressing the limitation of the proposed method in future work.

## CRedit authorship contribution statement

**Caie Hu:** Code, Investigation, Methodology, Writing – original draft, Validation. **Sanyou Zeng:** Project administration, Resources, Methodology, Supervision. **Changhe Li:** Funding acquisition, Supervision, Methodology, Writing – review & editing.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

No data was used for the research described in the article.

## Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grants 62076226, in part by the Fundamental Research Funds for the Central Universities China University of Geosciences (Wuhan) under Grant CUGGC02, in part by the 111 project under Grant B17040, in part by the PhD research startup foundation of Xi'an University of Technology under Grant 103-256082312.

## References

- [1] H. Ismikh, Black box optimization using evolutionary algorithm with novel selection and replacement strategies based on similarity between solutions, *Appl. Soft Comput.* 64 (2018) 260–271.
- [2] J.H. Holland, *Adaptation in Natural and Artificial Systems*, University of Michigan Press, 1975.
- [3] I. Rechenberg, *Evolution Strategiesoptimierung Technischer Systeme Nach Prinzipien Der Biologischen Evolution*, Frommann-Holzboog, 1973.
- [4] R. Storn, K. Price, Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces, *J. Global Optim.* 11 (4) (1997) 341–359.
- [5] R. Eberhart, J. Kennedy, A new optimizer using particle swarm theory, in: *Proceedings of the Sixth International Symposium on Micro Machine and Human Science*, 1995, pp. 39–43.
- [6] M.S. Mavec, J. Brest, A review of the recent use of differential evolution for large-scale global optimization: An analysis of selected algorithms on the CEC 2013 LSGO benchmark suite, *Swarm Evol. Comput.* 50 (2019), 100428.
- [7] B. Liu, Q. Zhang, G.G.E. Gielen, A Gaussian process surrogate model assisted evolutionary algorithm for medium scale expensive optimization problems, *IEEE Trans. Evol. Comput.* 18 (2) (2014) 180–192.
- [8] Y. Jin, Surrogate-assisted evolutionary computation: Recent advances and future challenges, *Swarm Evol. Comput.* 1 (2) (2011) 61–70.
- [9] Y. Jin, A comprehensive survey of fitness approximation in evolutionary computation, *Soft Comput.* 9 (1) (2005) 3–12.
- [10] D. Lim, Y. Jin, Y.-S. Ong, B. Sendhoff, Generalizing surrogate-assisted evolutionary computation, *IEEE Trans. Evol. Comput.* 14 (3) (2009) 329–355.
- [11] Y. Jin, H. Wang, T. Chugh, D. Guo, K. Miettinen, Data-driven evolutionary optimization: an overview and case studies, *IEEE Trans. Evol. Comput.* 23 (3) (2018) 442–458.
- [12] W. Wang, H.-L. Liu, K.C. Tan, A surrogate-assisted differential evolution algorithm for high-dimensional expensive optimization problems, *IEEE Trans. Cybern.* (2022) 1–13, <https://doi.org/10.1109/TCYB.2022.3175533>.
- [13] Q. Zhang, W. Liu, T. Edward, V. Botond, Expensive multiobjective optimization by MOEA/D with Gaussian process model, *IEEE Trans. Evol. Comput.* 14 (3) (2010) 456–474.
- [14] Y. Liu, Z. Yang, D. Xu, H. Qiu, L. Gao, A surrogate-assisted differential evolution for expensive constrained optimization problems involving mixed-integer variables, *Inform. Sci.* 622 (2023) 282–302.
- [15] N. Hansen, A global surrogate assisted CMA-ES, in: *2019 Genetic and Evolutionary Computation Conference*, 2019, pp. 664–672.
- [16] D.R. Jones, M. Schonlau, W.J. Welch, Efficient global optimization of expensive black-box functions, *J. Global Optim.* 13 (4) (1998) 455–492.
- [17] V. Torczon, M.W. Trosset, Using approximations to accelerate engineering design optimization, in: *1998 7th AIAA/USAF/NASA/ISSMO Symposium on Multidisciplinary Analysis and Optimization*, 1998, pp. 1–15.
- [18] H. Ulmer, F. Streichert, A. Zell, Evolution strategies assisted by Gaussian processes with improved preselection criterion, in: *2003 IEEE Congress on Evolutionary Computation*, 2003, pp. 692–699.
- [19] Q. Liu, R. Cheng, Y. Jin, M. Heiderich, T. Rodemann, Reference vector-assisted adaptive model management for surrogate-assisted many-objective optimization,

- IEEE Trans. Syst. Man Cybern.: Syst. (2022) 1–14, <https://doi.org/10.1109/TSMC.2022.3163129>.
- [20] Z. Zhou, Y.-S. Ong, M.H. Nguyen, D. Lim, A study on polynomial regression and Gaussian process global surrogate model in hierarchical surrogate-assisted evolutionary algorithm, in: IEEE Congress on Evolutionary Computation, 2005, pp. 2832–2839.
- [21] S. Matthias, Gaussian processes for machine learning, *Int. J. Neural Syst.* 14 (02) (2004) 69–106.
- [22] R.G. Regis, Evolutionary programming for high-dimensional constrained expensive black-box optimization using radial basis functions, *IEEE Trans. Evol. Comput.* 18 (3) (2013) 326–347.
- [23] I. Loshchilov, M. Schoenauer, M. Sebag, Self-adaptive surrogate-assisted covariance matrix adaptation evolution strategy, in: 2012 Genetic and Evolutionary Computation Conference, 2012, pp. 321–328.
- [24] Y. Jin, B. Sendhoff, Reducing fitness evaluations using clustering techniques and neural network ensembles, in: 2004 Genetic and Evolutionary Computation Conference, 2004, pp. 688–699.
- [25] G. Brown, J.L. Wyatt, P. Tino, Y. Bengio, Managing diversity in regression ensembles, *J. Mach. Learn. Res.* 6 (9) (2005) 1621–1650.
- [26] D. Huang, T.T. Allen, W.I. Notz, N. Zeng, Global optimization of stochastic black-box systems via sequential kriging meta-models, *J. Global Optim.* 34 (3) (2006) 441–466.
- [27] D. Han, W. Du, X. Wang, W. Du, A surrogate-assisted evolutionary algorithm for expensive many-objective optimization in the refining process, *Swarm Evol. Comput.* 69 (2022) (2022) 1–15.
- [28] B. Liu, Q. Zhang, G. Gielen, A surrogate-model-assisted evolutionary algorithm for computationally expensive design optimization problems with inequality constraints, in: *Simulation-Driven Modeling and Optimization*, Springer, 2016, pp. 347–370.
- [29] X. Fan, K. Li, K.C. Tan, Surrogate assisted evolutionary algorithm based on transfer learning for dynamic expensive multi-objective optimisation problems, in: 2020 IEEE Congress on Evolutionary Computation, 2020, pp. 1–8.
- [30] H. Liu, Y.-S. Ong, X. Shen, J. Cai, When Gaussian process meets big data: a review of scalable GPs, *IEEE Trans. Neural Netw. Learn. Syst.* 31 (11) (2020) 4405–4423.
- [31] N.D. Lawrence, J.C. Platt, Learning to learn with the informative vector machine, in: *Proceedings of the Twenty-First International Conference on Machine Learning*, 2004, pp. 1–8.
- [32] E.V. Bonilla, K. Chai, C. Williams, Multi-task Gaussian process prediction, in: *Proceedings of the 20th Annual Conference on Neural Information Processing Systems*, 2008, pp. 153–160.
- [33] J. Gao, W. Fan, J. Jiang, J. Han, Knowledge transfer via multiple model local structure mapping, in: *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2008, pp. 283–291.
- [34] J.M. Bernardo, J.O. Berger, A.P. Dawid, A.F.M. Smith, Regression and classification using Gaussian process priors (with discussion), *Bayesian Stat.* 6 (1998) 475–501.
- [35] C.E. Rasmussen, Gaussian processes in machine learning, in: *Summer School on Machine Learning*, 2003, pp. 63–71.
- [36] M.T. Emmerich, K.C. Giannakoglou, B. Naujoks, Single-and multiobjective evolutionary optimization assisted by Gaussian random field metamodels, *IEEE Trans. Evol. Comput.* 10 (4) (2006) 421–439.
- [37] C. Hu, S. Zeng, C. Li, F. Zhao, On nonstationary Gaussian process model for solving data-driven optimization problems, *IEEE Trans. Cybern.* (2021) 1–14, <https://doi.org/10.1109/TCYB.2021.3120188>.
- [38] J. MacQueen, Some methods for classification and analysis of multivariate observations, in: 1967 the Fifth Berkeley Symposium on Mathematical Statistics and Probability, 1967, pp. 281–297.
- [39] D. Buche, N.N. Schraudolph, P. Koumoutsakos, Accelerating evolutionary algorithms with Gaussian process fitness function models, *IEEE Trans. Syst. Man Cybern. C* 35 (2) (2005) 183–194.
- [40] S.J. Pan, Q. Yang, A survey on transfer learning, *IEEE Trans. Knowl. Data Eng.* 22 (10) (2009) 1345–1359.
- [41] K. Weiss, T.M. Khoshgoftaar, D. Wang, A survey of transfer learning, *J. Big Data* 3 (1) (2016) 1–40.
- [42] A. Gupta, Y.-S. Ong, L. Feng, Insights on transfer optimization: Because experience is the best teacher, *IEEE Trans. Emerg. Top. Comput. Intell.* 2 (1) (2017) 51–64.
- [43] Q. Yang, Y. Zhang, W. Dai, S.J. Pan, *Transfer Learning*, Cambridge University Press, 2020.
- [44] L. Shao, F. Zhu, X. Li, Transfer learning for visual categorization: A survey, *IEEE Trans. Neural Netw. Learn. Syst.* 26 (5) (2014) 1019–1034.
- [45] A. Gupta, Y.-S. Ong, L. Feng, Multifactorial evolution: toward evolutionary multitasking, *IEEE Trans. Evol. Comput.* 20 (3) (2015) 343–357.
- [46] A. Abbas, M.M. Abdelsamea, M.M. Gaber, 4S-DT: Self-supervised super sample decomposition for transfer learning with application to COVID-19 detection, *IEEE Trans. Neural Netw. Learn. Syst.* 32 (7) (2021) 2798–2808.
- [47] W. Li, R. Huang, J. Li, Y. Liao, Z. Chen, G. He, R. Yan, K. Gryllias, A perspective survey on deep transfer learning for fault diagnosis in industrial scenarios: Theories, applications and challenges, *Mech. Syst. Signal Process.* 167 (2022) (2022) 1–30.
- [48] S. Kullback, R.A. Leibler, On information and sufficiency, *Ann. Math. Stat.* 22 (1) (1951) 79–86.
- [49] M.L. Menéndez, J.A. Pardo, L. Pardo, M.C. Pardo, The jensen-shannon divergence, *J. Franklin Inst.* B 334 (2) (1997) 307–318.
- [50] C. Villani, *The wasserstein distances*, in: *Optimal Transport*, Springer, 2009, pp. 93–111.
- [51] M. Stein, Large sample properties of simulations using Latin hypercube sampling, *Technometrics* 29 (2) (1987) 143–151.
- [52] B. Liu, Q. Chen, Q. Zhang, J. Liang, P.N. Suganthan, B. Qu, Problem definitions and evaluation criteria for computational expensive optimization, in: 2014 IEEE Congress on Evolutionary Computation, 2014, pp. 2081–2088.
- [53] C. Hu, S. Zeng, C. Li, Hyperparameters adaptive sharing based on transfer learning for scalable GPs, in: 2022 IEEE Congress on Evolutionary Computation, 2022, pp. 1–8.
- [54] K.V. Price, *Differential evolution*, in: *Handbook of Optimization*, Springer, 2013, pp. 187–214.
- [55] J.H. Zar, in: *Biostatistical Analysis*, fifth ed., Prentice-Hall, Upper Saddle River, NJ, USA, 2007.